

实验三、为 Unix V6++ 系统添加新的系统调用

同济大学计算机系操作系统
2351520

2023 计算机拔尖班
毛星博

1. 为 Unix V6++ 添加 49# 系统调用 `get_ppid`，获得父进程的 `pid` 号

(1) 修改系统调用表 `m_SystemEntranceTable`，增加 49 号系统调用 `Sys_Getppid()`

```
/* 49 = getppid    count = 0    */  
static int Sys_Getppid();
```

(2) 修改 `m_SystemEntranceTable` 的第 49 个表项

`Sys_Getppid` (不需要任何参数，因为只需要访问当前进程控制块中的 `p_ppid` 字段，所以 `count = 0`)

```
SystemCallTableEntry SystemCall::m_SystemEntranceTable[SYSTEM_CALL_NUM] =  
{  
    ...  
    { 0, &Sys_Getppid },          /* 49 = getppid */  
    ...  
};
```

(3) 编写 `Sys_Getppid` 函数，实现获得父进程的 `pid` 号的功能

```
/* 49 = getppid    count = 0    */  
int SystemCall::Sys_Getppid()  
{  
    User& u = Kernel::Instance().GetUser();  
  
    /* 获取当前进程的父进程 PID */  
    u.u_ar0[User::EAX] = u.u_procp->p_ppid;  
  
    return 0; /* GCC likes it ! */  
}
```

(4) 在 `sys.c` 文件添加新系统调用的钩子函数实现

```
/* 49 号系统调用 - 获取父进程 PID */  
int getppid()  
{  
    int res;  
    __asm__ volatile("int $0x80" : "=a"(res) : "a"(49));  
    if (res >= 0)  
        return res;  
    return -1;  
}
```

(5) 在 `sys.h` 文件添加新系统调用的函数声明

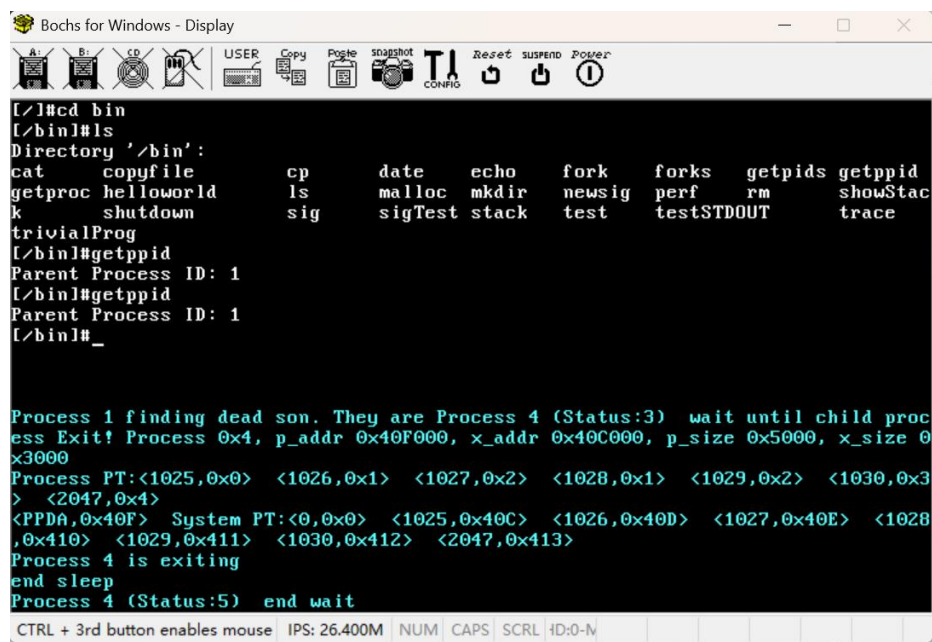
```
/* 获取父进程 PID */  
int getppid();
```

(6) 编写新系统调用的测试程序 (在 `makefile` 中添加构建目标和编译规则)

```
int main1(int argc, char *argv[])
```

```
{
    // 调用 getppid() 获取父进程 PID
    int ppid = getppid();
    // 显示父进程 PID
    printf("Parent Process ID: %d\n", ppid);
    return 0;
}
```

如图，在 bochs 虚拟机中运行 getppid 程序，显示父进程 ID 为 1。这是因为系统启动时，首先创建 0# 进程（系统进程），0# 进程创建 1# 进程，1# 进程执行 shell 程序。在 shell 中执行 getppid 命令时，shell 进程通过 fork() 创建一个新的子进程执行 getppid 命令。因此，getppid 命令的父进程就是 shell 进程，PID 为 1。



```
Bochs for Windows - Display
[/]#cd bin
[/bin]#ls
Directory '/bin':
cat      copyfile  cp      date      echo      fork      forks      getpids  getppid
getproc  helloworld ls       malloc    mkdir     newsig    perf      rm       showStac
k        shutdown  sig      sigTest   stack     test      testSTDOUT trace
trivialProg
[/bin]#getppid
Parent Process ID: 1
[/bin]#getppid
Parent Process ID: 1
[/bin]#_

Process 1 finding dead son. They are Process 4 (Status:3) wait until child process Exit! Process 0x4, p_addr 0x40F000, x_addr 0x40C000, p_size 0x5000, x_size 0x3000
Process PT:<1025,0x0> <1026,0x1> <1027,0x2> <1028,0x1> <1029,0x2> <1030,0x3> <2047,0x4>
<PPDA,0x40F> System PT:<0,0x0> <1025,0x40C> <1026,0x40D> <1027,0x40E> <1028,0x410> <1029,0x411> <1030,0x412> <2047,0x413>
Process 4 is exiting
end sleep
Process 4 (Status:5) end wait
CTRL + 3rd button enables mouse IPS: 26.400M NUM CAPS SCRL ID:0-N
```

2. 为 Unix V6++ 添加 50# 系统调用 get_pids，获得自己 和 父进程的 pid 号

(1) 修改系统调用表 m_SystemEntranceTable，增加 50 号系统调用 Sys_Getpids()

```
/* 50 = getpids count = 1 */
static int Sys_Getpids();
```

(2) 修改 m_SystemEntranceTable 的第 50 个表项

这个系统调用用于同时获取当前进程 PID 和父进程 PID

需要 1 个参数：一个指向整数数组的指针(pids)，所以 count = 1

用户程序预先分配足够空间的数组来接收这两个 PID 值

系统调用将当前进程 PID 存入 pids[0]，父进程 PID 存入 pids[1]

```
SystemCallTableEntry SystemCall::m_SystemEntranceTable[SYSTEM_CALL_NUM] =
{
    ...
    { 1, &Sys_Getpids },          /* 50 = getpids */
    ...
};
```

(3) 编写 Sys_Getpids() 函数，实现获得自己 和 父进程的 pid 号的功能

```

/* 50 = getpid    count = 1 */
int SystemCall::Sys_Getpids()
{
    User& u = Kernel::Instance().GetUser();

    /* 获取参数：指向一个结构的指针，用于存储当前进程和父进程的 PID */
    int* pids = (int*)u.u_arg[0];

    if (pids == NULL)
    {
        u.u_error = User::EFAULT;
        return 0;
    }

    /* 第一个整数：当前进程 PID */
    pids[0] = u.u_procp->p_pid;
    /* 第二个整数：父进程 PID */
    pids[1] = u.u_procp->p_ppid;

    /* 成功返回 2（返回两个 PID 值） */
    u.u_ar0[User::EAX] = 2;

    return 0; /* GCC likes it ! */
}

```

(4) 在 sys.c 文件添加新系统调用的钩子函数实现

```

/* 50 号系统调用 - 获取当前进程和父进程 PID 数组 */
int getpid(pid_t *pids)
{
    int res;
    __asm__ volatile("int $0x80" : "=a"(res) : "a"(50), "b"(pids));
    if (res >= 0)
        return res;
    return -1;
}

```

(5) 在 sys.h 文件添加新系统调用的函数声明

```

/* 获取当前进程和父进程 PID 数组 */
int getpid(pid_t *pids);

```

(6) 编写新系统调用的测试程序（在 makefile 中添加构建目标和编译规则）

```

int main1(int argc, char *argv[])
{
    int pids[2]; // 数组用于存储当前进程和父进程 PID

    // 调用 getpid() 获取 PID 数组
    int result = getpid(pids);
}

```

```

if (result == 2)
{
    // 显示获取到的PID信息
    printf("Current Process ID: %d\n", pids[0]);
    printf("Parent Process ID: %d\n", pids[1]);
}
else
{
    printf("Error getting process IDs\n");
}
return 0;
}

```

如图，在 bochs 虚拟机中多次运行 `getpids` 程序，显示父进程 ID 始终为 1，当前进程 ID 从 2 开始不断递增。这是因为，系统使用 `ProcessManager` 类中的静态成员变量 `m_NextUniquePid`（初始值为 0）来跟踪下一个要分配的进程 ID。分配进程 ID 的函数 `NextUniquePid()` 每次调用都会返回当前值并递增。

```

Bochs for Windows - Display
[/]#cd bin
[/bin]#getpids
Current Process ID: 2
Parent Process ID: 1
[/bin]#getpids
Current Process ID: 3
Parent Process ID: 1
[/bin]#getpids
Current Process ID: 4
Parent Process ID: 1
[/bin]#_

Process 1 finding dead son. They are Process 4 (Status:3) wait until child process Exit! Process 0x4, p_addr 0x40F000, x_addr 0x40C000, p_size 0x5000, x_size 0x3000
Process PT:<1025,0x0> <1026,0x1> <1027,0x2> <1028,0x1> <1029,0x2> <1030,0x3> <2047,0x4>
<PPDA,0x40F> System PT:<0,0x0> <1025,0x40C> <1026,0x40D> <1027,0x40E> <1028,0x410> <1029,0x411> <1030,0x412> <2047,0x413>
Process 4 is exiting
end sleep
Process 4 (Status:5) end wait
CTRL + 3rd button enables mouse | IPS: 28.800M | NUM | CAPS | SCRL | ID:0-N

```

3.为 Unix V6++添加 51#系统调用 `get_proc`，获得进程 `Process` 结构和 `User` 结构中的多个字段，比如：

虚空间：代码段起始地址，代码段长度，数据段起始地址，数据段长度，堆栈段长度

物理空间：代码段起始地址，可交换部分起始地址

(1) 修改系统调用表 `m_SystemEntranceTable`，增加 51 号系统调用 `Sys_Getproc()`

```

/* 51 = getproc count = 1 */
static int Sys_Getproc();

```

(2) 修改 `m_SystemEntranceTable` 的第 51 个表项

这个系统调用用于获取进程的详细信息

需要 1 个参数：一个指向 `ProcessInfo` 结构体的指针，所以 `count = 1`

用户程序提供 `ProcessInfo` 结构体来接收进程的完整信息

系统调用会填充该结构体，包括虚拟空间信息、物理空间信息和进程基本信息

```
SystemCallTableEntry SystemCall::m_SystemEntranceTable[SYSTEM_CALL_NUM] =
{
    ...
    { 1, &Sys_Getproc },          /* 51 = getproc */
    ...
};
```

(3) 编写 Sys_Getppid 函数，获得进程 Process 结构和 User 结构中的虚拟空间信息、物理空间信息和进程基本信息

```
/* 51 = getproc count = 1 */
int SystemCall::Sys_Getproc()
{
    User& u = Kernel::Instance().GetUser();

    /* 获取参数：指向进程信息结构体的指针 */
    // 定义一个结构体来存储进程信息
    struct ProcessInfo {
        // 虚拟空间信息
        unsigned long textStart;    // 代码段起始地址
        unsigned long textSize;    // 代码段长度
        unsigned long dataStart;   // 数据段起始地址
        unsigned long dataSize;    // 数据段长度
        unsigned long stackSize;   // 堆栈段长度

        // 物理空间信息
        unsigned long textPhyStart; // 代码段起始地址（物理）
        unsigned long swapStart;    // 可交换部分起始地址（物理）

        // 进程基本信息
        int pid;                    // 进程 ID
        int ppid;                   // 父进程 ID
        int state;                  // 进程状态
    };

    ProcessInfo* procInfo = (ProcessInfo*)u.u_arg[0];

    if (procInfo == NULL)
    {
        u.u_error = User::EFAULT;
        return 0;
    }

    // 获取内存描述符
    MemoryDescriptor& memDesc = u.u_MemoryDescriptor;
```

```

// 填充虚拟空间信息
procInfo->textStart = memDesc.GetTextStartAddress();
procInfo->textSize = memDesc.GetTextSize();
procInfo->dataStart = memDesc.GetDataStartAddress();
procInfo->dataSize = memDesc.GetDataSize();
procInfo->stackSize = memDesc.GetStackSize();

// 填充物理空间信息
// 代码段物理起始地址 = p_addr (这个是页对齐的物理地址)
procInfo->textPhyStart = u.u_procp->p_addr;
// 可交换部分起始地址 = p_addr + 代码段大小 (按页对齐)
procInfo->swapStart = u.u_procp->p_addr + memDesc.GetTextSize();

// 填充进程基本信息
procInfo->pid = u.u_procp->p_pid;
procInfo->ppid = u.u_procp->p_ppid;
procInfo->state = u.u_procp->p_stat;

/* 成功返回 0 */
u.u_ar0[User::EAX] = 0;

return 0; /* GCC likes it ! */
}

```

(4) 在 sys.c 文件添加新系统调用的钩子函数实现

```

/* 51 号系统调用 - 获取进程详细信息 */
int getproc(struct proc_info *info)
{
    int res;
    __asm__ volatile("int $0x80" : "=a"(res) : "a"(51), "b"(info));
    if (res >= 0)
        return res;
    return -1;
}

```

(5) 在 sys.h 文件添加新系统调用的函数声明

```

/* 获取进程详细信息 */
int getproc(struct proc_info *info);

```

以及 **proc_info** 结构体定义，包含进程的虚拟空间信息、物理空间信息和基本信息

```

/* 进程信息结构体定义 */
struct proc_info
{
    /* 虚拟空间信息 */
    unsigned long textStart; /* 代码段起始地址 */
    unsigned long textSize; /* 代码段长度 */

```

```

unsigned long dataStart; /* 数据段起始地址 */
unsigned long dataSize; /* 数据段长度 */
unsigned long stackSize; /* 堆栈段长度 */

/* 物理空间信息 */
unsigned long textPhyStart; /* 代码段起始地址（物理） */
unsigned long swapStart; /* 可交换部分起始地址（物理） */

/* 进程基本信息 */
int pid; /* 进程 ID */
int ppid; /* 父进程 ID */
int state; /* 进程状态 */
};

```

(6) 编写新系统调用的测试程序（在 makefile 中添加构建目标和编译规则）

```

int main1(int argc, char *argv[])
{
    struct proc_info info;

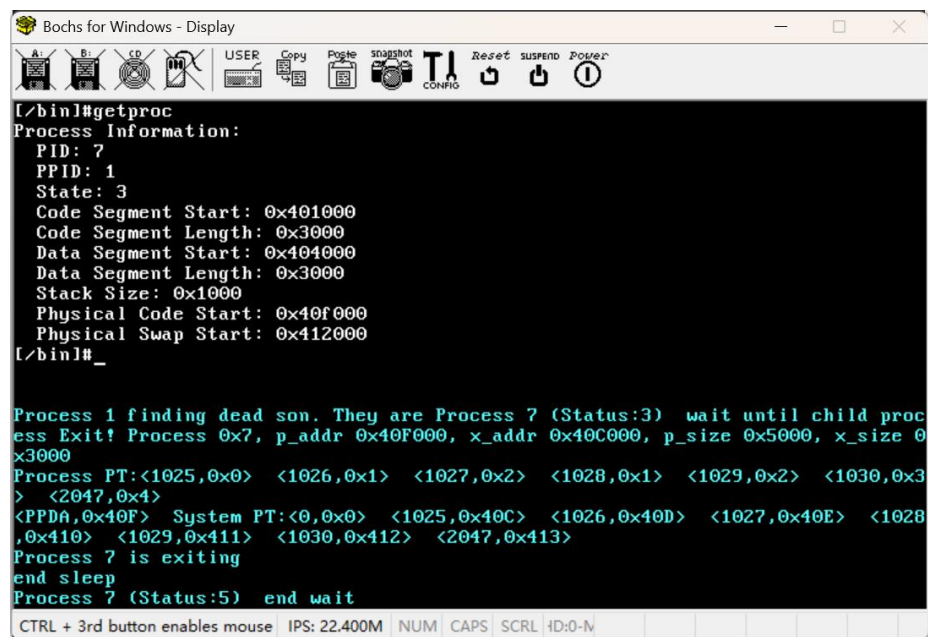
    // 调用 getproc() 获取进程详细信息
    int result = getproc(&info);
    if (result == 0)
    {
        // 显示获取到的进程信息
        printf("Process Information:\n");
        printf("  PID: %d\n", info.pid);
        printf("  PPID: %d\n", info.ppid);
        printf("  State: %d\n", info.state);
        printf("  Code Segment Start: 0x%lx\n", info.textStart);
        printf("  Code Segment Length: 0x%lx\n", info.textSize);
        printf("  Data Segment Start: 0x%lx\n", info.dataStart);
        printf("  Data Segment Length: 0x%lx\n", info.dataSize);
        printf("  Stack Size: 0x%lx\n", info.stackSize);
        printf("  Physical Code Start: 0x%lx\n", info.textPhyStart);
        printf("  Physical Swap Start: 0x%lx\n", info.swapStart);
    }
    else
    {
        printf("Error getting process information\n");
    }
    return 0;
}

```

如图，在 bochs 虚拟机中运行 getproc() 程序，显示进程的虚拟空间信息、物理空间信息和基本信息：

PID: 7 ：当前进程的唯一标识符为 7，由系统通过递增计数器分配

PPID: 1 : 父进程 ID 为 1, 说明这个 `getproc` 进程是由 `shell` 进程 (PID=1) 创建的
State: 3 : 进程当前状态码为 3, `SRUN` 代表进程正在运行
Code Segment Start: 0x401000 : 代码段在虚拟地址空间的起始地址为 0x401000
Code Segment Length: 0x3000 : 代码段长度为 3 页
Data Segment Start: 0x404000 : 数据段在虚拟地址空间的起始地址为 0x404000
Data Segment Length: 0x3000 : 数据段长度为 3 页
Stack Size: 0x1000 : 堆栈段大小为 1 页
Physical Code Start: 0x40F000 : 代码段在物理内存中的起始地址为 0x40F000
Physical Swap Start: 0x412000 : 可交换部分在物理内存中的起始地址为 0x412000



```
Bochs for Windows - Display
[ /bin]#getproc
Process Information:
  PID: 7
  PPID: 1
  State: 3
  Code Segment Start: 0x401000
  Code Segment Length: 0x3000
  Data Segment Start: 0x404000
  Data Segment Length: 0x3000
  Stack Size: 0x1000
  Physical Code Start: 0x40f000
  Physical Swap Start: 0x412000
[ /bin]#

Process 1 finding dead son. They are Process 7 (Status:3) wait until child process Exit! Process 0x7, p_addr 0x40f000, x_addr 0x40c000, p_size 0x5000, x_size 0x3000
Process PT:<1025,0x0> <1026,0x1> <1027,0x2> <1028,0x1> <1029,0x2> <1030,0x3> <2047,0x4>
<PPDA,0x40f> System PT:<0,0x0> <1025,0x40c> <1026,0x40d> <1027,0x40e> <1028,0x410> <1029,0x411> <1030,0x412> <2047,0x413>
Process 7 is exiting
end sleep
Process 7 (Status:5) end wait
CTRL + 3rd button enables mouse IPS: 22.400M NUM| CAPS SCRL 4D:0-IV
```

4、以最后一小题（题 3）为背景，评论 CISC、RISC 架构的优势和不足

CISC（复杂指令集计算机）：

优势：

1. 丰富的指令集：CISC 提供了丰富的指令，包括特权指令和复杂操作，使得系统调用实现可以更简洁。像 Intel x86 架构中的 `int 0x80` 中断指令就能直接实现系统调用切换，指令语义完整。
2. 内存访问灵活性：CISC 架构支持复杂的内存寻址模式，便于在系统调用中高效地在内核空间和用户空间之间传输数据结构（如 `get_proc` 需要获取的进程信息结构体）。
3. 硬件支持系统调用：CISC 通常有专门的系统调用指令（如 `syscall`），硬件直接支持权限切换，简化了内核态/用户态的转换过程。
4. 代码密度高：一条指令可以完成多个操作，使得系统调用的实现代码更紧凑，占用内存少。

不足：

1. 实现复杂：指令执行周期不固定，导致 CPU 设计复杂，不利于超标量和流水线优化。
2. 上下文切换开销大：复杂的寄存器组 and 状态保存机制增加了系统调用的上下文切换时间。
3. 难以并行：指令间依赖关系复杂，难以在系统调用处理中充分利用并行计算能力。

RISC（精简指令集计算机）：

优势：

1. 流水线效率高：RISC 指令长度固定、执行周期统一，流水线执行效率高，系统调用的基本操作速度快。
2. 上下文切换轻量：寄存器窗口设计（如 SPARC）使得系统调用时的上下文保存和恢复更加高效。
3. 并行处理能力强：简单指令更容易实现超标量和并行执行，提高系统调用的处理效率。
4. 设计简洁：CPU 设计更简洁，有利于提高时钟频率和整体性能。

不足：

1. 指令数量多：完成复杂操作需要更多指令，系统调用实现代码可能更长。在 `get_proc` 这样需要获取多个字段的系统调用中，可能需要更多的内存传输指令。
2. 需要软件辅助：许多复杂功能需要通过软件组合实现，系统调用的设计可能更复杂。
3. 内存访问模式受限：简化的寻址模式可能需要更多指令来完成复杂的数据传输操作，影响系统调用的执行效率。

理清系统调用执行过程：

1. 用户空间调用阶段

- 用户程序通过调用 C 语言接口函数（如 `getpids`、`getproc` 等）发起系统调用。这些函数在 `sys.c` 中实现，核心操作是使用内联汇编指令触发中断。
- 系统调用号被存入 `EAX` 寄存器，参数（如 `info` 指针）被存入 `EBX` 等寄存器。通过 `int $0x80` 指令触发中断，请求进入内核态

2. 中断处理与上下文切换阶段

- 触发中断后，CPU 执行以下操作：
- 保存当前用户空间上下文（包括寄存器状态）
- 切换到内核态执行权限

跳转到系统调用入口函数 `SystemCallEntrance`

3. 系统调用分发阶段

- 在 `Trap` 函数中：从 `EAX` 寄存器获取系统调用号；从用户空间参数区域获取其他参数；在系统调用表 `m_SystemEntranceTable` 中查找对应的处理函数

4. 系统调用处理阶段

- 执行对应的系统调用处理函数：
- 访问内核数据结构（如 `ProcessManager`、`MemoryDescriptor` 等）
- 实现具体功能逻辑（如获取进程信息）
- 设置返回值到 `EAX` 寄存器
- 设置错误码

5. 返回用户空间阶段

- 恢复用户空间上下文
- 从内核态切换回用户态
- 控制权返回给用户程序
- 用户程序从 `EAX` 寄存器获取系统调用结果